In [12]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [13]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, r
```

In [14]:
```
df = pd.read_csv("DataSets/diabetes.csv")

df.head()
```

Out[14]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeF
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

In [15]:
```
print("Shape of dataset:", df.shape)

print("\nColumn names:")
print(df.columns)

print("\nMissing values:")
print(df.isnull().sum())
```

```
Shape of dataset: (768, 9)

Column names:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')

Missing values:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

In [17]:
```
df.shape
```

Out[17]:
```
(768, 9)
```

In [18]:
```
df.columns
```

Out[18]:
```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')
```

In [19]:
```
df.isnull().sum()
```

Out[19]:
```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

In [20]:
```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X.head()
```

Out[20]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeF
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

In [21]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (614, 8)
Testing data: (154, 8)
```

In [22]:
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [23]:
```
k = 5

knn = KNeighborsClassifier(n_neighbors=k)

knn.fit(X_train, y_train)
```

Out[23]:

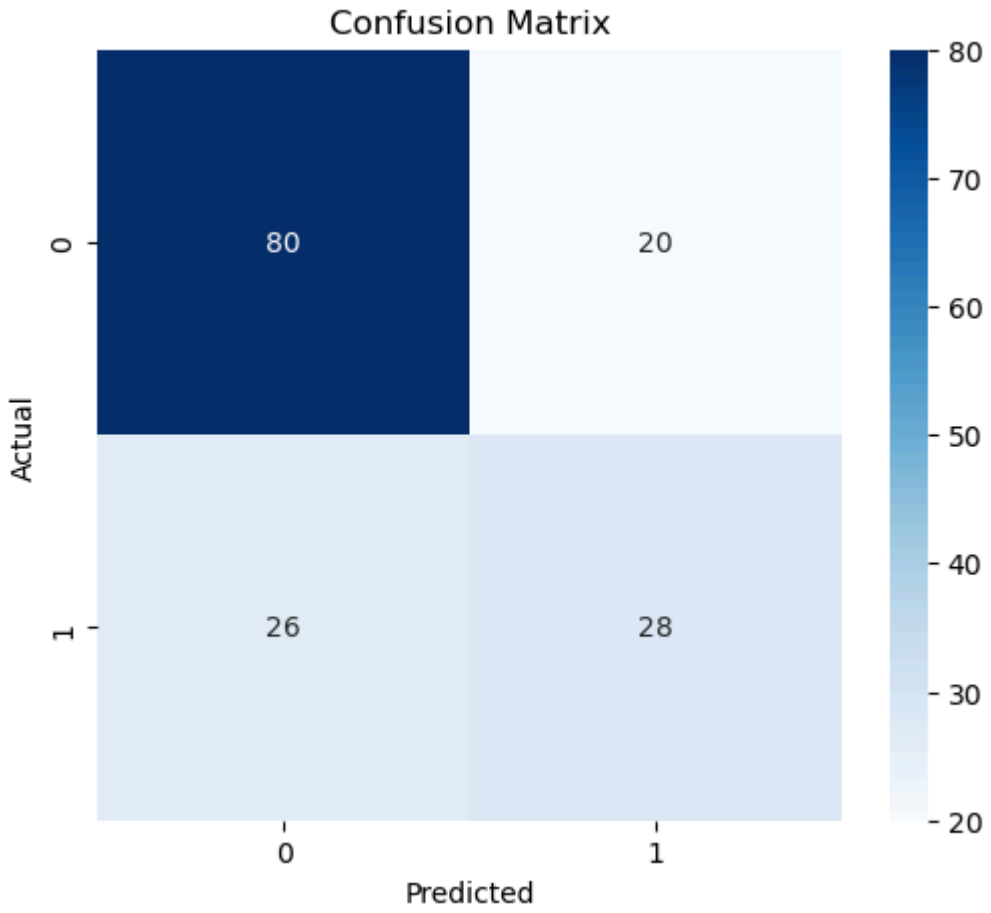
KNeighborsClassifier

KNeighborsClassifier()

In [24]:
```
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```



In [25]:
```
accuracy = accuracy_score(y_test, y_pred)

accuracy
```

Out[25]: 0.7012987012987013

In [26]:
```
error_rate = 1 - accuracy

error_rate
```

Out[26]: 0.2987012987012987

In [27]:
```
precision = precision_score(y_test, y_pred)

precision
```

Out[27]: 0.5833333333333334

In [28]:
```
recall = recall_score(y_test, y_pred)

recall
```

Out[28]: 0.5185185185185185

In [29]:
```
results = pd.DataFrame({
    "Metric": ["Accuracy", "Error Rate", "Precision", "Recall"],
    "Value": [accuracy, error_rate, precision, recall]
})

results
```

Out[29]:

	Metric	Value
0	Accuracy	0.701299
1	Error Rate	0.298701
2	Precision	0.583333
3	Recall	0.518519

In [30]:
```
results["Percentage"] = results["Value"] * 100

results
```

Out[30]:

	Metric	Value	Percentage
0	Accuracy	0.701299	70.129870
1	Error Rate	0.298701	29.870130
2	Precision	0.583333	58.333333
3	Recall	0.518519	51.851852

In [ ]:
```

```